

Software Verification Plan – StudyQuest

Titel des Dokuments:

Software Verification Plan – StudyQuest

Projektname:

StudyQuest – Gamifizierte Lern- und Notenverwaltungs-Web-App

Modul:

Software Engineering I – Praxis

Projektzeitraum:

Wintersemester 2025 / 2026

Version:

1.3

Datum:

28. Februar 2026

Author:innen:

- Michael Steer (Scrum Master)
- Luke Engehardt (Product Owner)
- Giuliana Carrano (Developer)
- Paul Strasser (Developer)
- Roman Faber (Developer)

Betreuer / Prüfer:

Sascha Wanninger

Freigabe durch autorisierte Person:

Michael Steer (Scrum Master)

Changelog

Version	Datum	Autor	Änderungsbeschreibung
---------	-------	-------	-----------------------

0.1	10.10.2025	M. Steer	Initiale Gliederung nach ECSS-Q-ST-80C erstellt
1.0	17.10.2025	M. Steer	Erstfassung des Software Verification Plans erstellt
1.1	30.10.2025	M. Steer	Entry/Exit Criteria präzisiert, Verifikationsmeth definiert
1.2	13.01.2026	M. Steer	Testfallkatalog um UC03–UC15 erweitert, Traceability-Matri angelegt
1.3	28.02.2026	M. Steer	Testumgebung aktualisiert, Smoke-Test ergänzt, Finalversion zur Abgabe vorbereitet

Distribution List

Name		
Sascha Wanninger		
Michael Steer		
Luke Engehardt		
Giuliana Carrano		
Paul Strasser		
Roman Faber		

1. Einleitung & Zielsetzung

Dieser Software Verification Plan (SVP) beschreibt, **wie** die StudyQuest-Web-App systematisch verifiziert wird, sodass die Implementierung in `src/` die spezifizierten Anforderungen erfüllt und zuverlässig funktioniert.

Verifikationsziel:

- Abdeckung der Anforderungen aus `Documents/Anforderungsanalyse/Requirements.md` (funktional + nicht-funktional).
- Nachweis der korrekten Umsetzung der 15 Use Cases (UC01–UC15).
- Sicherstellung von Nachvollziehbarkeit (Traceability) zwischen Anforderungen ↔ Testfällen ↔ Ergebnissen.

2. Anwendungsbereich

2.1 Verifikationsobjekt (Software Item)

- **Typ:** Client-side Web App (HTML/CSS/JavaScript)
- **Persistenz:** Browser LocalStorage

- **Entry Point:** `src/index.html`
- **Wesentliche Module:**
- Presentation: `ui.js`, `index.html`, `css/*`
- Business Logic: `user.js`, `quest.js`, `grade.js`, `leaderboard.js`, `achievement.js`, `admin.js`, `auth.js`
- Data Layer: `db.js`

2.2 In Scope / Out of Scope

In Scope - Funktionale Umsetzung UC01–UC15 inkl. Datenpersistenz im LocalStorage - UI-Workflows (Happy Path + negative Tests) - Performance-Ziele aus Requirements (z. B. Dashboard $\leq 2s$) - Zugriffsschutz (Login/Session, Admin-Rechte) soweit in Client-Architektur umsetzbar

Out of Scope / Einschränkungen - „Serverseitige“ Anforderungen (z. B. 401 API-Calls, serverseitige Logs) sind in der reinen Client-Architektur nur **äquivalent** über lokale Audit-Trails testbar. - Kryptografisch starkes Password Hashing (bcrypt) ist im Prototyp ggf. nicht umgesetzt; wird als **Security-Deviation** dokumentiert, wenn Acceptance Criteria nicht erfüllt werden. - E-Mail/Push-basierte Flows (Passwort-Reset-Mail, Push-Reminder) sind ohne Backend/Provider nicht end-to-end verifizierbar.

2.3 Codeabgleich (Stand: 28.02.2026)

Die Verifikation wurde mit dem aktuellen Code in `10_Durchfuehrung/src/` abgeglichen. Folgende Punkte sind als bekannte Abweichungen markiert und werden im SVP als Deviation/INS statt als erfolgreicher Systemtest geführt:

- **UC02 (Passwort-Reset):** nicht implementiert (kein Reset-UI, kein Token-Flow, kein Mailversand).
- **UC06-F3:** Timer-Pause/Fortsetzen nicht implementiert (vorhanden: Start/Stop, Ein-Timer-Regel).
- **UC06-F7:** Reconnect/Sync-Pufferlogik nicht implementiert.
- **UC09-F1..F3:** keine Benachrichtigungs-Einstellungen/Reminder-Toggles; implementiert ist Notification-Anzeige mit Read/Clear.
- **UC09-NF2:** keine Push-/E-Mail-Zustellung.
- **UC13-NF3/NF4, UC14-NF2, UC15-NF2:** keine Versionierungs-/Auditlog-Mechanik im Persistenzmodell.
- **UC07-F4, UC08-F5:** Notendurchschnitt/Notenstatistik im Dashboard sind in v1.0 nicht umgesetzt.

4. Verifikationsstrategie

4.1 Verifikationsmethoden (ECSS-Style)

Methode	Kürzel	Beschreibung	Beispiele in StudyQuest
Test	TST	Ausführen der Software und Vergleich mit Sollverhalten	UC-Workflows im Browser, Timer/Quest/Note
Analyse	ANL	Messung/Bewertung ohne vollständige Ausführung	Ladezeiten, Timer-Abweichung CSV Performance
Inspektion	INS	Review von Artefakten (Code/Doku)	Passwortspeicher Rollenchecks, Doku-Konsistenz
Demonstration	DEM	Geführtes Durchklicken zur Funktionsdemonstration	Smoke-Test, Admin-Panel Demo

4.2 Testlevel

- **Unit-ish Tests (optional):** Business-Logik-Funktionen in Isolation (sofern Testharness ergänzt wird).
- **Integration Tests:** Zusammenspiel Business-Logik ↔ Data Layer (LocalStorage) ↔ UI.
- **System Tests (Schwerpunkt):** End-to-End Use Case Tests im Browser.

4.3 Entry-/Exit-Kriterien

Entry - Requirements baseline ist freigegeben (Version 1.x). - Build/Start der App funktioniert via Live Server. - Testdaten/Accounts vorhanden (oder LocalStorage zurückgesetzt).

Exit - Alle „Muss“-Anforderungen haben mindestens 1 verknüpften Testfall und ein Ergebnis. - Abweichungen sind als Defects/Deviations dokumentiert (inkl. Impact & Entscheidung). - Testreport inkl. Traceability-Matrix liegt vor.

5. Verifikationsumgebung

5.1 Hardware/OS

- Standard-Entwickler-Laptop/PC
- Windows 11 / macOS / Linux (mindestens 1 Plattform, ideal 2)

5.2 Software

- Browser: Chrome (stable), Firefox (stable), Safari (macOS)
- VS Code + Live Server (oder äquivalenter static web server)
- Optional Automation: Node.js (≥ 18), Playwright, Jest, ESLint

5.3 Testdaten

- Default Quests via `DB.init()`
- Test-User:
- `test@example.com / password123` (falls genutzt)

- Admin: erster registrierter User (laut Implementierung)

6. Testfallkatalog

6.1 Übersicht

TC	Title	Level	Method	Related
TC-GEN-01	Smoke-Test: App startet & DB wird initialisiert	System	DEM/TST	UC08-F1, UC08-NF2
TC-UC01-01	Registrierung erfolgreich (neue E-Mail)	System	TST	UC01-F1
TC-UC01-02	Registrierung verhindert Duplikat-E-Mail	System	TST	UC01-F2
TC-UC01-03	Login (gültig/ungültig) & Dashboard-Navigation	System	TST	UC01-F3, UC01-F4

TC-UC01-NF-01	Security: Passwort nicht im Klartext + neutrale Fehlermeldungen	System	INS/TST	UC01-NF1, UC01-NF2
TC-UC02-01	Passwort-Reset Feature-Check (nicht implementiert)	System	INS	UC02-F1, UC02-NF1 (Deviation)
TC-UC02-02	Reset-Token/Ablaufzeit Feature-Check (nicht implementiert)	System	INS	UC02-F2, UC02-NF2 (Deviation)
TC-UC02-03	Passwortwechsel-Flow Feature-Check (nicht implementiert)	System	INS	UC02-F3, UC02-F4 (Deviation)
TC-UC03-01	Profil anzeigen	System	TST	UC03-F1
TC-UC03-02	Profil ändern: Name/Avatar speichern & wieder anzeigen	System	TST	UC03-F2, UC03-F3

TC-UC03-NF-01	Zugriffsschutz & Fehlermeldungen bei Profiländerung	System	TST	UC03-NF1, UC03-NF2
TC-UC04-01	Quest auswählen & starten (Status + Startzeit)	System	TST	UC04-F1, UC04-F2, UC04-F3
TC-UC04-02	Nur eine aktive Quest: Start einer zweiten Quest blockiert	System	TST	UC04-F4, UC04-F5
TC-UC04-NF-01	Performance: Statuswechsel beim Start $\leq$ 1s	System	ANL/TST	UC04-NF1
TC-UC05-01	Quest abschließen: XP-Vergabe, Endzeit, Status	System	TST	UC05-F1, UC05-F2, UC05-F5, UC05-NF4

TC-UC05-02	Atomare XP-Vergabe: Doppelklick/Mehrfach-Submit	System	TST	UC05-NF2
TC-UC05-03	Level-Up: Schwellenwert & visuelles Feedback	System	TST	UC05-F3, UC05-F4
TC-UC05-NF-01	Performance: Abschlussreaktion $\leq$ 2s	System	ANL/TST	UC05-NF1
TC-UC06-01	Timer starten: Startzeit & Persistenz	System	TST	UC06-F1, UC06-F2
TC-UC06-02	Ein-Timer-Regel (Pause/Fortsetzen nicht implementiert)	System	TST/INS	UC06-F6, UC06-F3 (Deviation)
TC-UC06-03	Timer stoppen: Dauerberechnung & optionale XP-Vergabe	System	TST	UC06-F4, UC06-F5

TC-UC06-04	Timer bleibt nach Reload erhalten	System	TST	UC06-NF2
TC-UC06-NF-01	Timer-Genauigkeit & Response-Time	System	ANL/TST	UC06-NF1, UC06-NF4
TC-UC07-01	Noten anlegen	System	TST	UC07-F1
TC-UC07-02	Noten bearbeiten & löschen	System	TST	UC07-F2, UC07-F3, UC07-F5, UC07-NF1
TC-UC07-03	Durchschnittsberechnung & Datenkonsistenz	System	TST	UC07-F4, UC07-NF2
TC-UC08-01	Dashboard zeigt korrekte Statistiken (XP/Level/Quests/Noten)	System	TST	UC08-F1, UC08-F2, UC08-F3, UC08-F4, UC08-F5
TC-UC08-NF-01	Dashboard Ladezeit $\leq$ 2s + Browser-Kompatibilität	System	ANL/TST	UC08-NF1, UC08-NF2

TC-UC09-01	In-App Notifications: anzeigen, gelesen markieren, alle gelesen	System	TST/INS	UC09-F4 (teilweise), UC09-F1..F3 (Deviation)
TC-UC09-NF-01	In-App Notification Reaktionsverhalten (keine Push/E-Mail)	System	TST/INS	UC09-NF1 (teilweise), UC09-NF2 (Deviation)
TC-UC10-01	CSV Export: Inhalt & Format korrekt	System	TST	UC10-F2
TC-UC10-02	CSV Import: gültige Datei	System	TST	UC10-F1
TC-UC10-03	CSV Import: ungültige Datei -> Fehlermeldung	System	TST	UC10-F3

TC-UC10-NF-01	Performance Import/Export ≤ 5s bei 100 Einträgen	System	ANL/TST	UC10-NF1
TC-UC11-01	Badges werden bei erfüllter Bedingung freigeschaltet	System	TST	UC11-F1, UC11-F2
TC-UC11-02	Visuelles Feedback + Badge-Übersicht	System	TST	UC11-F3, UC11-F4
TC-UC11-NF-01	Automatischer Badge-Check nach relevanten Aktionen	System	TST	UC11-NF1
TC-UC12-01	Leaderboard: Ranking in 4 Kategorien + persönliche Position	System	TST	UC12-F1, UC12-F4
TC-UC12-02	Streak-Berechnung & Aktualisierung	System	TST	UC12-F2, UC12-F3

TC-UC12-NF-01	Performance: Leaderboard ≤ 3s	System	ANL/TST	UC12-NF1
TC-UC13-01	Admin Login/Access: Admin sieht Admin Panel, Non-Admin nicht	System	TST	UC13-F1, UC13-NF2
TC-UC13-02	Quest CRUD im Admin Panel (Create/Update/Delete)	System	TST	UC13-F2, UC13-F3, UC13-F4, UC13-F5, UC13-F6
TC-UC13-NF-01	Admin Änderungen sichtbar ≤ 2s	System	ANL/TST	UC13-NF1
TC-UC14-01	Game Rules ändern & sofort wirksam	System	TST	UC14-F1, UC14-F2, UC14-F3, UC14-F4, UC14-F5

TC-UC14-NF-01	Sicherheit & Performance beim Speichern (Auditlog Deviation)	System	TST/ANL/INS	UC14-NF3, UC14-NF4, UC14-NF2 (Deviation)
TC-UC15-01	User-Administration: Admin-Rechte, Aktiv/Deaktiv, Löschen	System	TST/INS	UC15-F1, UC15-F4, UC15-F5, UC15-F2/F3 (Deviation)
TC-UC15-NF-01	Zugriffsschutz + Reaktionszeit ≤ 2s (ohne Auditlog)	System	TST/ANL/INS	UC15-NF1, UC15-NF3, UC15-NF2 (Deviation)

7.2 Detaillierte Testfallbeschreibungen

Hinweis: Testfälle mit Kennzeichnung **(Deviation)** werden im aktuellen Projektstand nicht als „bestanden“ im Sinne einer Feature-Implementierung gewertet, sondern als dokumentierte Lücke zwischen Requirement und v1.0-Codebasis (Inspection-Nachweis). Alle Testfälle folgen einer einheitlichen Struktur: Testlevel, Methode, verifizierte Anforderungen, Vorbedingungen, Schritte, erwartetes Ergebnis/Pass-Kriterium.

TC-GEN-01 – Smoke-Test: App startet & DB wird initialisiert

- **Testlevel:** System

- **Methode:** DEM/TST
- **Verifizierte Anforderungen:** UC08-F1, UC08-NF2
- **Vorbedingungen:** Live Server läuft; Browser Cache leer; LocalStorage für Domain leer.

Schritte 1) src/index.html via Live Server öffnen 2) DevTools Console prüfen 3) Seite neu laden (Ctrl+R) 4) LocalStorage Keys prüfen

Erwartetes Ergebnis / Pass-Kriterium App lädt ohne JS-Fehler; Navbar abhängig vom Login-Status; DB.init legt Default-Stores an (users, quests, learning_sessions, ...).

TC-UC01-01 – Registrierung erfolgreich (neue E-Mail)

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC01-F1

- **Vorbedingungen:** App geöffnet; keine User mit der Test-E-Mail existieren.

Schritte 1) Registrieren-Tab öffnen 2) Name, E-Mail, Passwort (≥ 6 Zeichen) eingeben 3) Absenden 4) LocalStorage 'users' prüfen

Erwartetes Ergebnis / Pass-Kriterium Neuer User-Datensatz wird angelegt; User ist eingeloggt; Dashboard wird angezeigt.

TC-UC01-02 – Registrierung verhindert Duplikat-E-Mail

- **Testlevel:** System

- **Methode:** TST

- **Verifizierte Anforderungen:** UC01-F2

- **Vorbedingungen:** Mindestens ein User mit der Test-E-Mail existiert.

Schritte 1) Erneut Registrieren mit gleicher E-Mail 2) Absenden

Erwartetes Ergebnis / Pass-Kriterium Registrierung wird abgelehnt;
Fehlermeldung erscheint; kein zweiter Datensatz entsteht.

TC-UC01-03 – Login (gültig/ungültig) & Dashboard-Navigation

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC01-F3, UC01-F4
- **Vorbedingungen:** Mindestens ein registrierter User vorhanden.

Schritte 1) Login-Tab öffnen 2) Korrekte E-Mail und Passwort eingeben → Absenden 3) Prüfen, ob Dashboard angezeigt wird 4) Logout → erneut Login-Tab öffnen 5) Korrekte E-Mail, falsches Passwort eingeben → Absenden

Erwartetes Ergebnis / Pass-Kriterium Login mit korrekten Daten zeigt Dashboard; falsches Passwort wird mit neutraler Fehlermeldung abgelehnt; kein Zugriff auf Dashboard ohne erfolgreichen Login.

TC-UC01-NF-01 – Security: Passwort-Speicherung & Fehlermeldungen

- **Testlevel:** System
- **Methode:** INS/TST
- **Verifizierte Anforderungen:** UC01-NF1, UC01-NF2
- **Vorbedingungen:** Mindestens ein registrierter User vorhanden.

Schritte 1) LocalStorage 'users' im Browser-DevTools inspizieren 2) Prüfen, ob password_hash-Feld kein Klartext-Passwort enthält 3) Login mit nicht-existierender E-Mail versuchen 4) Login mit existierender E-Mail aber falschem Passwort versuchen 5) Fehlermeldungen beider Fälle vergleichen

Erwartetes Ergebnis / Pass-Kriterium Passwort ist in password_hash nicht im Klartext erkennbar; Fehlermeldungen bei nicht-existierender E-Mail und falschem Passwort sind identisch/neutral (kein Hinweis auf Existenz des Accounts).

TC-UC02-01 – Passwort-Reset Feature-Check (v1.0 Deviation) **[Kurzform - Deviation Example]**

- **Testlevel:** System
- **Methode:** INS

- **Verifizierte Anforderungen:** UC02-F1, UC02-NF1

- **Vorbedingungen:** Aktuelle App-Version v1.0 gestartet.

Schritte 1) Auth-UI prüfen (Login/Register) 2) Nach 'Passwort vergessen'/Reset-Einstieg suchen 3) `auth.js` auf Reset-Handler prüfen

Erwartetes Ergebnis / Pass-Kriterium Reset-Flow ist in v1.0 nicht vorhanden; Abweichung wird dokumentiert.

Vollständige Testfallbeschreibungen (UC03–UC15)

TC-UC03-01 – Profil anzeigen

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC03-F1
- **Vorbedingungen:** User ist eingeloggt; Profildaten sind gespeichert.

Schritte 1) Zur Profilseite navigieren 2) Angezeigte Daten (Name, Avatar, Level, XP) mit LocalStorage-Werten vergleichen

Erwartetes Ergebnis / Pass-Kriterium Alle Profildaten werden korrekt aus der DB geladen und im UI angezeigt.

TC-UC03-02 – Profil ändern: Name/Avatar speichern & wieder anzeigen

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC03-F2, UC03-F3
- **Vorbedingungen:** User ist eingeloggt.

Schritte 1) Zur Profilseite navigieren 2) Name ändern (z.B. „Max Mustermann“ → „Neuer Name“) 3) Avatar ändern (z.B. anderes Emoji wählen) 4) Speichern klicken 5) Seite neu laden und Profilseite erneut öffnen

Erwartetes Ergebnis / Pass-Kriterium Geänderte Daten werden gespeichert und nach Reload korrekt angezeigt; Erfolgsmeldung erscheint nach Speichern.

TC-UC03-NF-01 – Zugriffsschutz & Fehlermeldungen bei Profiländerung

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC03-NF1, UC03-NF2
- **Vorbedingungen:** Kein User eingeloggt (Session gelöscht).

Schritte 1) Direkt Profilseite aufrufen (ohne Login) 2) Prüfen, ob Redirect zum Login erfolgt 3) Einloggen → Profilseite öffnen → leeren Namen eingeben → Speichern

Erwartetes Ergebnis / Pass-Kriterium Ohne Login: Redirect zum Login-Formular. Bei ungültigen Eingaben: verständliche Fehlermeldung.

TC-UC04-01 – Quest auswählen & starten (Status + Startzeit)

- **Testlevel:** System

- **Methode:** TST
- **Verifizierte Anforderungen:** UC04-F1, UC04-F2, UC04-F3
- **Vorbedingungen:** User eingeloggt; keine aktive Quest; mindestens eine Quest verfügbar.

Schritte 1) Quest-Übersicht öffnen 2) Verfügbare Quest auswählen und „Starten“ klicken 3) Quest-Status im UI und LocalStorage prüfen 4) Startzeitpunkt im User-Datensatz (`active_quest_id`) prüfen

Erwartetes Ergebnis / Pass-Kriterium Quest-Status wechselt auf „aktiv“; `active_quest_id` ist gesetzt; Startzeitpunkt ist protokolliert.

TC-UC04-02 – Nur eine aktive Quest: Start einer zweiten Quest blockiert

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC04-F4, UC04-F5

- **Vorbedingungen:** User hat bereits eine aktive Quest.

Schritte 1) Quest-Übersicht öffnen 2) Andere verfügbare Quest auswählen und „Starten“ versuchen

Erwartetes Ergebnis / Pass-Kriterium Fehlermeldung „Eine Quest ist bereits aktiv“ erscheint; zweite Quest wird NICHT gestartet; `active_quest_id` bleibt unverändert.

TC-UC04-NF-01 – Performance: Statuswechsel beim Start $\leq 1s$

- **Testlevel:** System

- **Methode:** ANL/TST

- **Verifizierte Anforderungen:** UC04-NF1

- **Vorbedingungen:** User eingeloggt; Quest verfügbar.

Schritte 1) DevTools Performance-Tab öffnen 2) Quest starten und Reaktionszeit messen

Erwartetes Ergebnis / Pass-Kriterium UI-Update (Quest-Status → aktiv)
erfolgt in ≤ 1 Sekunde nach Klick.

TC-UC05-01 – Quest abschließen: XP-Vergabe, Endzeit, Status

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC05-F1, UC05-F2, UC05-F5, UC05-NF4
- **Vorbedingungen:** User hat eine aktive Quest.

Schritte 1) XP-Stand vor Abschluss notieren 2) „Quest abschließen“ klicken 3) XP-Stand danach prüfen (Differenz = Quest-XP) 4) Quest-Status in LocalStorage prüfen 5) LearningSession-Eintrag mit Endzeitpunkt prüfen

Erwartetes Ergebnis / Pass-Kriterium XP werden korrekt addiert gemäß Schwierigkeit (easy=50, medium=100, hard=150); Quest-Status = „completed“; `completed_at` Timestamp gespeichert; `active_quest_id` = null.

TC-UC05-02 – Atomare XP-Vergabe: Doppelklick/Mehrfach-Submit

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC05-NF2
- **Vorbedingungen:** User hat eine aktive Quest; XP-Stand notiert.

Schritte 1) „Quest abschließen“-Button schnell doppelklicken 2) XP-Stand prüfen

Erwartetes Ergebnis / Pass-Kriterium XP werden exakt einmal gutgeschrieben (keine Doppel-XP); Button wird nach erstem Klick deaktiviert oder zweiter Klick wird ignoriert.

TC-UC05-03 – Level-Up: Schwellenwert & visuelles Feedback

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC05-F3, UC05-F4

- **Vorbedingungen:** User XP knapp unter Level-Schwellwert (z.B. 480 XP bei 500er-Schwelle); aktive Quest mit ausreichend XP.

Schritte 1) Quest abschließen (XP pushes über Schwellwert) 2) Level-Anzeige prüfen 3) Visuelles Feedback (Popup/Banner) beobachten

Erwartetes Ergebnis / Pass-Kriterium Level erhöht sich um 1; visuelles Level-Up-Feedback wird angezeigt; überschüssige XP werden im neuen Level korrekt berechnet.

TC-UC05-NF-01 – Performance: Abschlussreaktion $\leq 2s$

- **Testlevel:** System
- **Methode:** ANL/TST
- **Verifizierte Anforderungen:** UC05-NF1
- **Vorbedingungen:** User hat aktive Quest.

Schritte 1) DevTools Performance-Tab öffnen 2) Quest abschließen und Gesamtreaktionszeit messen (Klick → Dashboard-Update)

Erwartetes Ergebnis / Pass-Kriterium Vollständiges UI-Update (XP, Level, Quest-Status) in ≤ 2 Sekunden.

TC-UC06-01 – Timer starten: Startzeit & Persistenz

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC06-F1, UC06-F2
- **Vorbedingungen:** User eingeloggt; aktive Quest vorhanden.

Schritte 1) „Timer starten“ klicken 2) Timer-Anzeige im UI beobachten (läuft der Zähler?) 3) LocalStorage timers-Eintrag prüfen (started_at vorhanden?)

Erwartetes Ergebnis / Pass-Kriterium Timer-UI zeigt laufende Zeit; Timer-Datensatz mit started_at Timestamp in DB gespeichert.

TC-UC06-02 – Ein-Timer-Regel (Pause/Fortsetzen nicht implementiert)

- **Testlevel:** System
- **Methode:** TST/INS
- **Verifizierte Anforderungen:** UC06-F6, UC06-F3 (Deviation)
- **Vorbedingungen:** Timer bereits aktiv.

Schritte 1) Erneut „Timer starten“ versuchen 2) Code-Inspektion: `quest.js` nach Pause/Resume-Logik durchsuchen

Erwartetes Ergebnis / Pass-Kriterium Fehlermeldung „Timer bereits aktiv“; kein zweiter Timer erstellt. Pause/Resume: nicht vorhanden → Deviation dokumentiert.

TC-UC06-03 – Timer stoppen: Dauerberechnung & optionale XP-Vergabe

- **Testlevel:** System
- **Methode:** TST

- **Verifizierte Anforderungen:** UC06-F4, UC06-F5

- **Vorbedingungen:** Timer läuft seit ≥ 2 Minuten.

Schritte 1) „Timer stoppen“ klicken 2) Angezeigte Dauer mit erwarteter Zeit (Stop – Start) vergleichen 3) XP-Vergabe prüfen: $XP_bonus = duration_minutes \times xp_per_minute_timer$ 4) LearningSession-Eintrag in LocalStorage prüfen

Erwartetes Ergebnis / Pass-Kriterium Korrekte Dauerberechnung; Timer-Bonus-XP werden optional vergeben; Session mit `duration_seconds` und `xp_earned` gespeichert.

TC-UC06-04 – Timer bleibt nach Reload erhalten

- **Testlevel:** System

- **Methode:** TST

- **Verifizierte Anforderungen:** UC06-NF2

- **Vorbedingungen:** Timer läuft.

Schritte 1) Browser-Tab neu laden (F5) 2) Timer-Anzeige prüfen

Erwartetes Ergebnis / Pass-Kriterium Timer läuft nach Reload weiter; angezeigte Zeit ist konsistent (keine Zurücksetzung).

TC-UC06-NF-01 – Timer-Genauigkeit & Response-Time

- **Testlevel:** System
- **Methode:** ANL/TST
- **Verifizierte Anforderungen:** UC06-NF1, UC06-NF4
- **Vorbedingungen:** Timer aktiv.

Schritte 1) Timer 5 Minuten laufen lassen 2) Angezeigte Zeit mit externer Stoppuhr vergleichen 3) Start/Stop-Reaktionszeit messen

Erwartetes Ergebnis / Pass-Kriterium Zeitabweichung ≤ 1 Minute über 5 Minuten; UI-Reaktion auf Start/Stop ≤ 1 Sekunde.

TC-UC07-01 – Noten anlegen

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC07-F1
- **Vorbedingungen:** User eingeloggt; Notenübersicht geöffnet.

Schritte 1) „Neue Note“ klicken 2) Modulname (z.B. „Analysis I“), Note (z.B. 2.3), Gewichtung eingeben 3) Speichern klicken 4) Notenübersicht und LocalStorage prüfen

Erwartetes Ergebnis / Pass-Kriterium Note erscheint in der Übersicht; Grade-Datensatz mit korrekten Werten in DB gespeichert.

TC-UC07-02 – Noten bearbeiten & löschen

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC07-F2, UC07-F3, UC07-F5, UC07-NF1

- **Vorbedingungen:** Mindestens eine Note vorhanden.

Schritte 1) Note bearbeiten (Notenwert ändern) → Speichern 2) Prüfen, ob geänderter Wert in Übersicht und DB korrekt ist 3) Note löschen 4) Prüfen, ob Note aus Übersicht und DB entfernt wurde

Erwartetes Ergebnis / Pass-Kriterium Bearbeitete Note wird korrekt aktualisiert; gelöschte Note verschwindet; UI aktualisiert sich ohne Seitenneuladen (UC07-NF1).

TC-UC07-03 – Durchschnittsberechnung & Datenkonsistenz

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC07-F4, UC07-NF2
- **Vorbedingungen:** Mehrere Noten mit unterschiedlichen Gewichtungen vorhanden.

Schritte 1) Noten eingeben: Analysis I = 2.0 (Gewicht 5), Programmieren = 1.3 (Gewicht 8) 2) Durchschnitt anzeigen lassen 3) Schnell hintereinander: Note bearbeiten, löschen, neu anlegen 4) Konsistenz der Daten prüfen

Erwartetes Ergebnis / Pass-Kriterium Gewichteter Durchschnitt korrekt berechnet; schnelle Operationen verursachen keine Inkonsistenzen. (Hinweis: UC07-F4 ggf. Deviation wenn Durchschnittsfunktion nicht implementiert.)

TC-UC08-01 – Dashboard zeigt korrekte Statistiken

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC08-F1, UC08-F2, UC08-F3, UC08-F4, UC08-F5
- **Vorbedingungen:** User mit abgeschlossenen Quests, XP, Level, Noten.

Schritte 1) Dashboard öffnen 2) XP-Stand mit `total_xp_earned` in LocalStorage vergleichen 3) Level mit berechneter Erwartung vergleichen ($\text{total_xp} / \text{level_threshold}$) 4) Angezeigte Quests-Anzahl mit `quest_history` Array-Länge vergleichen 5) Notenstatistik prüfen (UC08-F5: ggf. Deviation dokumentieren)

Erwartetes Ergebnis / Pass-Kriterium Alle Werte (XP, Level, Quests, Badges) stimmen mit DB überein; Dashboard ist nach Login direkt sichtbar

(UC08-F1).

TC-UC08-NF-01 – Dashboard Ladezeit $\leq 2s$ + Browser-Kompatibilität

- **Testlevel:** System
- **Methode:** ANL/TST
- **Verifizierte Anforderungen:** UC08-NF1, UC08-NF2
- **Vorbedingungen:** App mit ~10 Quests, ~5 Users in LocalStorage.

Schritte 1) Chrome: Dashboard laden, Ladezeit mit DevTools messen 2) Firefox: Dashboard laden, Darstellung prüfen 3) Safari (macOS): Dashboard laden, Darstellung prüfen

Erwartetes Ergebnis / Pass-Kriterium Ladezeit ≤ 2 Sekunden in allen Browsern; Layout korrekt; keine JS-Fehler in Console.

TC-UC09-01 – In-App Notifications: anzeigen, gelesen markieren

- **Testlevel:** System

- **Methode:** TST/INS
- **Verifizierte Anforderungen:** UC09-F4 (teilweise), UC09-F1..F3 (Deviation)
- **Vorbedingungen:** User eingeloggt; Benachrichtigungen existieren (z.B. nach Quest-Abschluss).

Schritte 1) Notification-Icon/Badge im UI prüfen 2) Benachrichtigungsliste öffnen 3) Einzelne Benachrichtigung als gelesen markieren 4) „Alle gelesen“ klicken 5) Code-Inspektion: notification.js nach Settings/Reminder-Toggles durchsuchen

Erwartetes Ergebnis / Pass-Kriterium Ungelesene Benachrichtigungen werden angezeigt; gelesen-Markierung persistiert; alle-gelesen funktioniert. Deviation: Benachrichtigungs-Einstellungen (UC09-F1..F3) und Push/E-Mail (UC09-NF2) nicht vorhanden.

TC-UC09-NF-01 – Notification Reaktionsverhalten

- **Testlevel:** System
- **Methode:** TST/INS
- **Verifizierte Anforderungen:** UC09-NF1 (teilweise), UC09-NF2 (Deviation)

- **Vorbedingungen:** Benachrichtigungen vorhanden.

Schritte 1) Read-Status ändern → prüfen ob UI sofort aktualisiert (ohne Reload) 2) Code-Inspektion: nach Push-/E-Mail-Zustellung suchen

Erwartetes Ergebnis / Pass-Kriterium Status-Änderungen wirken sofort im UI (UC09-NF1). Push/E-Mail: nicht vorhanden (Deviation).

TC-UC10-01 – CSV Export: Inhalt & Format korrekt

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC10-F2
- **Vorbedingungen:** Mehrere Noten angelegt.

Schritte 1) „Noten exportieren“ klicken 2) CSV-Datei herunterladen und in Editor/Excel öffnen 3) Spalten und Werte mit DB-Daten vergleichen

Erwartetes Ergebnis / Pass-Kriterium Export-Datei enthält alle Noten mit korrekten Werten; CSV-Format ist valide (Komma-Separator, Header-Zeile).

TC-UC10-02 – CSV Import: gültige Datei

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC10-F1
- **Vorbedingungen:** Gültige CSV-Datei vorbereitet (z.B. vorher exportierte Datei).

Schritte 1) „Noten importieren“ klicken → CSV-Datei auswählen 2) Import bestätigen 3) Notenübersicht auf importierte Einträge prüfen

Erwartetes Ergebnis / Pass-Kriterium Alle Zeilen aus CSV werden als Grade-Datensätze importiert; Werte stimmen mit CSV überein.

TC-UC10-03 – CSV Import: ungültige Datei → Fehlermeldung

- **Testlevel:** System

- **Methode:** TST
- **Verifizierte Anforderungen:** UC10-F3
- **Vorbedingungen:** Ungültige CSV-Datei (z.B. falsche Spalten, leere Datei, .txt-Datei).

Schritte 1) „Noten importieren“ → ungültige Datei auswählen 2) Import versuchen

Erwartetes Ergebnis / Pass-Kriterium Verständliche Fehlermeldung erscheint; keine Daten werden importiert; bestehende Noten bleiben unverändert.

TC-UC10-NF-01 – Performance Import/Export ≤ 5s

- **Testlevel:** System
- **Methode:** ANL/TST
- **Verifizierte Anforderungen:** UC10-NF1

- **Vorbedingungen:** 100 Noten-Einträge in DB oder CSV.

Schritte 1) Export mit 100 Einträgen durchführen → Zeit stoppen 2) Import einer CSV mit 100 Einträgen durchführen → Zeit stoppen

Erwartetes Ergebnis / Pass-Kriterium Beide Operationen in ≤ 5 Sekunden abgeschlossen.

TC-UC11-01 – Badges werden bei erfüllter Bedingung freigeschaltet

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC11-F1, UC11-F2
- **Vorbedingungen:** User hat noch keine Badges; Bedingung für mindestens ein Badge ist erfüllbar (z.B. erste Quest abschließen).

Schritte 1) Ausgangszustand: Badge-Übersicht zeigt Badge als „gesperrt“ 2) Bedingung erfüllen (z.B. Quest abschließen → „Quest Starter“-Badge) 3) Badge-Übersicht erneut prüfen

Erwartetes Ergebnis / Pass-Kriterium Badge wechselt von „gesperrt“ zu „freigeschaltet“; Achievement-Eintrag in LocalStorage vorhanden.

TC-UC11-02 – Visuelles Feedback + Badge-Übersicht

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC11-F3, UC11-F4
- **Vorbedingungen:** Badge wird gerade freigeschaltet (siehe TC-UC11-01).

Schritte 1) Nach Badge-Freischaltung UI beobachten (Popup/Toast/Animation?) 2) Badge-Übersicht öffnen 3) Prüfen: Freigeschaltete Badges mit Icon/Titel sichtbar; gesperrte Badges ebenfalls sichtbar (ausgegraut)

Erwartetes Ergebnis / Pass-Kriterium Visuelles Feedback bei Freischaltung; Übersicht zeigt alle Badges (freigeschaltet + gesperrt).

TC-UC11-NF-01 – Automatischer Badge-Check nach relevanten Aktionen

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC11-NF1
- **Vorbedingungen:** User führt Badge-relevante Aktionen aus.

Schritte 1) Quest abschließen → prüfen ob Badge-Check getriggert wird 2) Timer beenden → prüfen ob Badge-Check getriggert wird 3) Console-Logs oder Achievement-Store auf Trigger-Einträge prüfen

Erwartetes Ergebnis / Pass-Kriterium Badge-Check wird automatisch nach jeder relevanten Aktion ausgeführt (nachweisbar via Logs oder Storage-Änderung).

TC-UC12-01 – Leaderboard: Ranking in 4 Kategorien + persönliche Position

- **Testlevel:** System
- **Methode:** TST

- **Verifizierte Anforderungen:** UC12-F1, UC12-F4
- **Vorbedingungen:** Mindestens 2 registrierte User mit unterschiedlichen XP/Level/Quests/Streaks.

Schritte 1) Leaderboard-Seite öffnen 2) Ranking nach XP prüfen (höchste XP zuerst?) 3) Andere Kriterien sortieren (Level, Quests, Streak) 4) Eigene Position suchen (hervorgehoben?)

Erwartetes Ergebnis / Pass-Kriterium Ranking ist korrekt sortiert in allen 4 Kategorien; eigene Position ist visuell hervorgehoben.

TC-UC12-02 – Streak-Berechnung & Aktualisierung

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC12-F2, UC12-F3
- **Vorbedingungen:** User mit aufeinanderfolgenden Tagen aktiver Nutzung.

Schritte 1) `last_activity_date` und `current_streak` im LocalStorage prüfen 2) Quest abschließen → Streak-Aktualisierung prüfen 3) Leaderboard öffnen → Streak-Wert prüfen

Erwartetes Ergebnis / Pass-Kriterium Streak wird korrekt berechnet (aufeinanderfolgende Tage); Leaderboard zeigt aktuellen Streak-Wert.

TC-UC12-NF-01 – Performance: Leaderboard ≤ 3s

- **Testlevel:** System
- **Methode:** ANL/TST
- **Verifizierte Anforderungen:** UC12-NF1
- **Vorbedingungen:** 5+ Users in DB.

Schritte 1) Leaderboard-Seite öffnen, Ladezeit mit DevTools messen

Erwartetes Ergebnis / Pass-Kriterium Vollständige Leaderboard-Anzeige in ≤ 3 Sekunden.

TC-UC13-01 – Admin Login/Access: Admin sieht Admin Panel, Non-Admin nicht

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC13-F1, UC13-NF2
- **Vorbedingungen:** Ein Admin-User und ein normaler User vorhanden.

Schritte 1) Als Admin einloggen → Admin-Panel-Link/Button prüfen 2) Admin-Panel öffnen → Zugriff erfolgreich? 3) Logout → als normaler User einloggen 4) Admin-Panel erreichbar?

Erwartetes Ergebnis / Pass-Kriterium Admin sieht und kann Admin-Panel nutzen; Non-Admin hat keinen Zugriff (kein Link sichtbar oder Redirect).

TC-UC13-02 – Quest CRUD im Admin Panel

- **Testlevel:** System

- **Methode:** TST
- **Verifizierte Anforderungen:** UC13-F2, UC13-F3, UC13-F4, UC13-F5, UC13-F6
- **Vorbedingungen:** Als Admin eingeloggt.

Schritte 1) Quest-Verwaltung öffnen → vorhandene Quests sehen 2) Neue Quest erstellen (Titel, Beschreibung, Schwierigkeit) 3) Erstelle Quest in Übersicht und DB prüfen 4) Quest bearbeiten (Titel ändern) → prüfen 5) Quest löschen → prüfen

Erwartetes Ergebnis / Pass-Kriterium CRUD-Operationen funktionieren; Änderungen sofort in der Übersicht sichtbar; DB-Einträge konsistent.

TC-UC13-NF-01 – Admin Änderungen sichtbar $\leq 2s$

- **Testlevel:** System
- **Methode:** ANL/TST
- **Verifizierte Anforderungen:** UC13-NF1

- **Vorbedingungen:** Als Admin eingeloggt; Quest-Verwaltung geöffnet.

Schritte 1) Quest erstellen/ändern/löschen → Zeit bis UI-Update messen

Erwartetes Ergebnis / Pass-Kriterium UI-Aktualisierung in ≤ 2 Sekunden nach jeder Aktion.

TC-UC14-01 – Game Rules ändern & sofort wirksam

- **Testlevel:** System
- **Methode:** TST
- **Verifizierte Anforderungen:** UC14-F1, UC14-F2, UC14-F3, UC14-F4, UC14-F5
- **Vorbedingungen:** Als Admin eingeloggt.

Schritte 1) Regelverwaltung öffnen → aktuelle Werte notieren 2) easy_xp von 50 auf 75 ändern → Speichern 3) LocalStorage game_rules prüfen (Wert = 75?) 4) Als normaler User Quest abschließen → prüfen ob 75 XP vergeben

werden

Erwartetes Ergebnis / Pass-Kriterium Geänderte Regeln werden gespeichert; neue Werte gelten sofort global für alle User.

TC-UC14-NF-01 – Sicherheit & Performance beim Speichern

- **Testlevel:** System
- **Methode:** TST/ANL/INS
- **Verifizierte Anforderungen:** UC14-NF3, UC14-NF4, UC14-NF2 (Deviation)
- **Vorbedingungen:** Als Admin eingeloggt.

Schritte 1) Regeln ändern → Speicherdauer messen ($\leq 2s$?) 2) Als Non-Admin versuchen, Regeln zu ändern (Code-Inspektion: Admin-Check vorhanden?) 3) Code-Inspektion: Nach Audit-Log für Regeländerungen suchen

Erwartetes Ergebnis / Pass-Kriterium Speichern ≤ 2 Sekunden; Zugriff nur für Admins; Audit-Log: nicht vorhanden (Deviation UC14-NF2).

TC-UC15-01 – User-Administration: Admin-Rechte, Aktiv/Deaktiv, Löschen

- **Testlevel:** System
- **Methode:** TST/INS
- **Verifizierte Anforderungen:** UC15-F1, UC15-F4, UC15-F5, UC15-F2/F3 (Deviation)
- **Vorbedingungen:** Als Admin eingeloggt; mindestens 2 User vorhanden.

Schritte 1) Admin-Panel öffnen → User-Liste prüfen 2) User deaktivieren (is_active = false) → als diesem User einloggen versuchen 3) User reaktivieren → Einloggen erneut versuchen 4) User löschen → prüfen ob aus DB entfernt 5) Code-Inspektion: Such-/Detailansicht vorhanden?

Erwartetes Ergebnis / Pass-Kriterium Deaktivierter User kann sich nicht einloggen; Reaktivierung stellt Zugang her; Löschung entfernt Datensatz. Suche/Detailansicht: ggf. vereinfacht (Deviation UC15-F2/F3).

TC-UC15-NF-01 – Zugriffsschutz + Reaktionszeit ≤ 2s

- **Testlevel:** System
- **Methode:** TST/ANL/INS

- **Verifizierte Anforderungen:** UC15-NF1, UC15-NF3, UC15-NF2 (Deviation)

- **Vorbedingungen:** Admin und Non-Admin User vorhanden.

Schritte 1) Als Non-Admin: Admin-Panel aufrufen versuchen → Zugriff verweigert? 2) Als Admin: User-Status ändern → Reaktionszeit messen 3) Code-Inspektion: Nach Audit-Log bei User-Aktionen suchen

Erwartetes Ergebnis / Pass-Kriterium Nur Admins können User verwalten; Statusänderung in ≤ 2 Sekunden; Audit-Log: nicht vorhanden (Deviation UC15-NF2).

8. Rollen & Verantwortlichkeiten (RACI-Light)

Aktivität	Product Owner	Scrum Master	Dev-Team	QA (rotierend)
Testfall-Design & Traceability	A	C	R	R
Durchführung Systemtests	C	C	R	R
Defect-Triage & Priorisierung	A	R	R	C

Fix + Re-Test	C	C	R	R
Abnahme (Muss-Anforderungen) ^A		C	R	C

A = Accountable, R = Responsible, C = Consulted

9. Zeitplan & Meilensteine

- **M1 – Testdesign fertig:** Testfälle + Traceability definiert
- **M2 – Systemtest 1 (Feature Complete):** UC01–UC12
- **M3 – Admin & Rules Tests:** UC13–UC15
- **M4 – Regression + Final Report:** Retest aller Defects + Abgabe Testreport

Hinweis: Für das Praxis-Modul kann der Zeitplan an Sprint-Termine angepasst werden.

10. Dokumentation, Reporting & Traceability

10.1 Testreporting

- Pro Testfall: Datum, Tester, Ergebnis (Pass/Fail), Evidence (Screenshot/Log), Defect-ID.
- Defects werden als GitHub Issues oder in einer zentralen Liste geführt (Severity, Repro-Steps, Fix-Version).

10.2 Rückverfolgbarkeit

Die vollständige Requirement-to-Test-Traceability ist als CSV hinterlegt:

- `SVP_StudyQuest_Traceability_Matrix.csv`